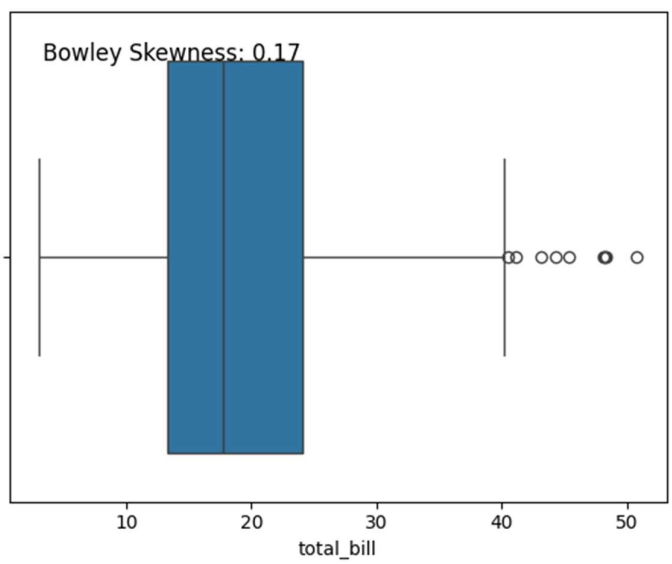
# 1.Bowley's Skewness

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
tips = sns.load_dataset('tips')
data = tips['total_bill']
Q1 = np.percentile(data, 25)
Q2 = np.percentile(data, 50)
Q3 = np.percentile(data, 75)
print(f"Q1: {Q1}, Q2: {Q2}, Q3: {Q3}")
B = (Q1 + Q3 - 2*Q2) / (Q3 - Q1)
sns.boxplot(x=data)
plt.annotate(f'Bowley Skewness: {B:.2f}', xy=(0.05, 0.9), xycoords='axes fraction',
fontsize=12)
plt.show()
```

OutPut:

```
Q1: 13.3475, Q2: 17.795, Q3: 24.127499999999998
```



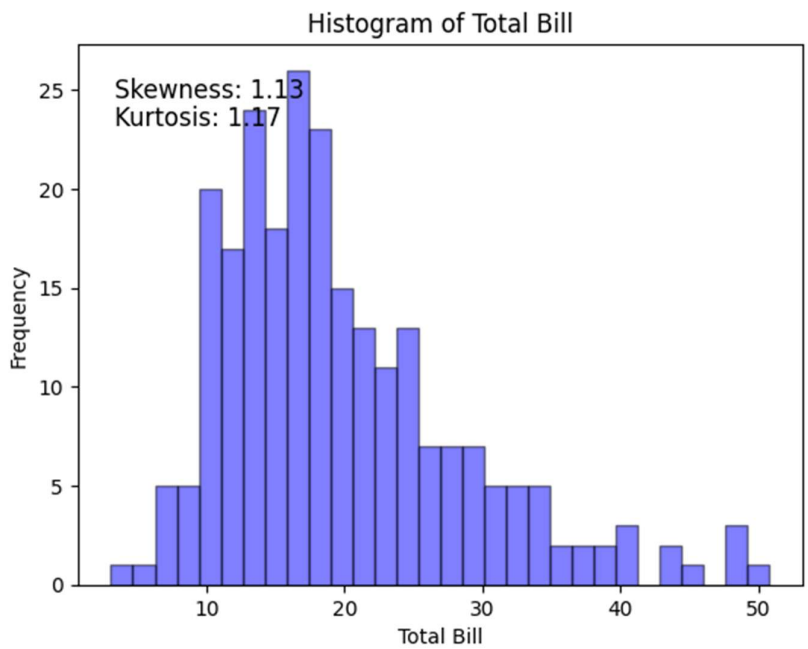

# 2. Skewness and Kurtosis

```
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import skew, kurtosis
tips = sns.load_dataset('tips')
data = tips['total_bill']
skewness = skew(data)
kurt = kurtosis(data)
```

```
plt.hist(data, bins=30, alpha=0.5, color='b', edgecolor='black')
plt.xlabel('Total Bill')
plt.ylabel('Frequency')
plt.title('Histogram of Total Bill')
plt.annotate(f'Skewness: {skewness:.2f}', xy=(0.05, 0.9), xycoords='axes fraction',
fontsize=12)
plt.annotate(f'Kurtosis: {kurt:.2f}', xy=(0.05, 0.85), xycoords='axes fraction',
fontsize=12)
plt.show()
print(f"Skewness: {skewness}")
print(f"Kurtosis: {kurt}")
```

Output:

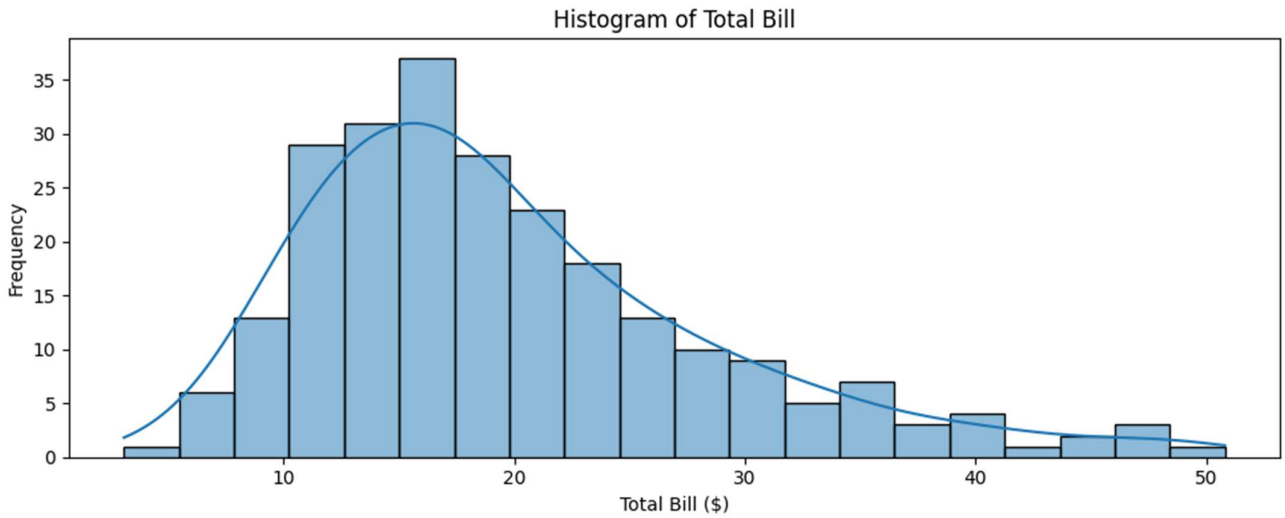
```
Skewness: 1.1262346334818638
Kurtosis: 1.1691681323851366
```

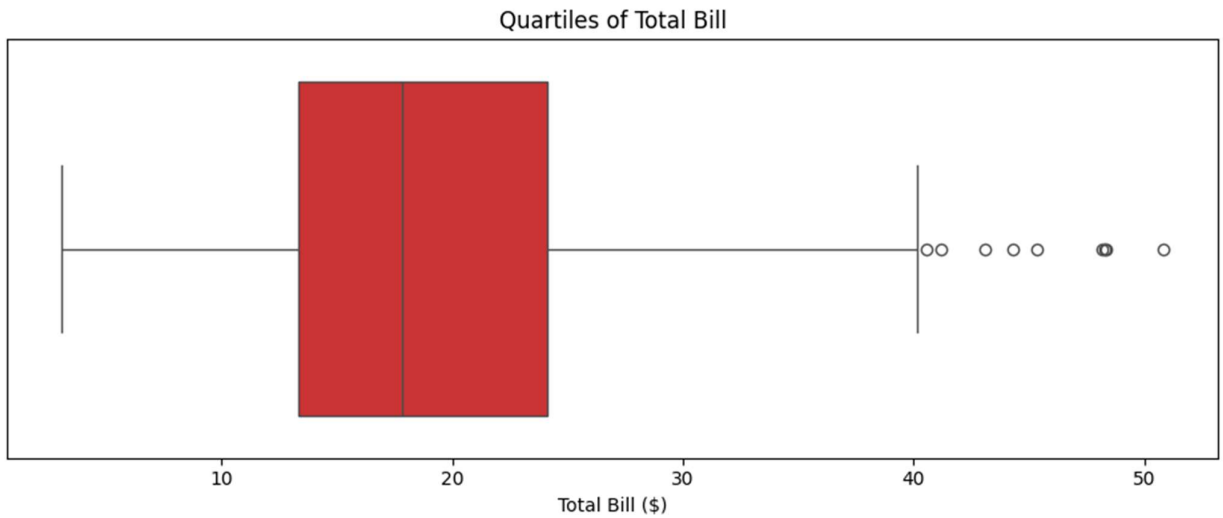


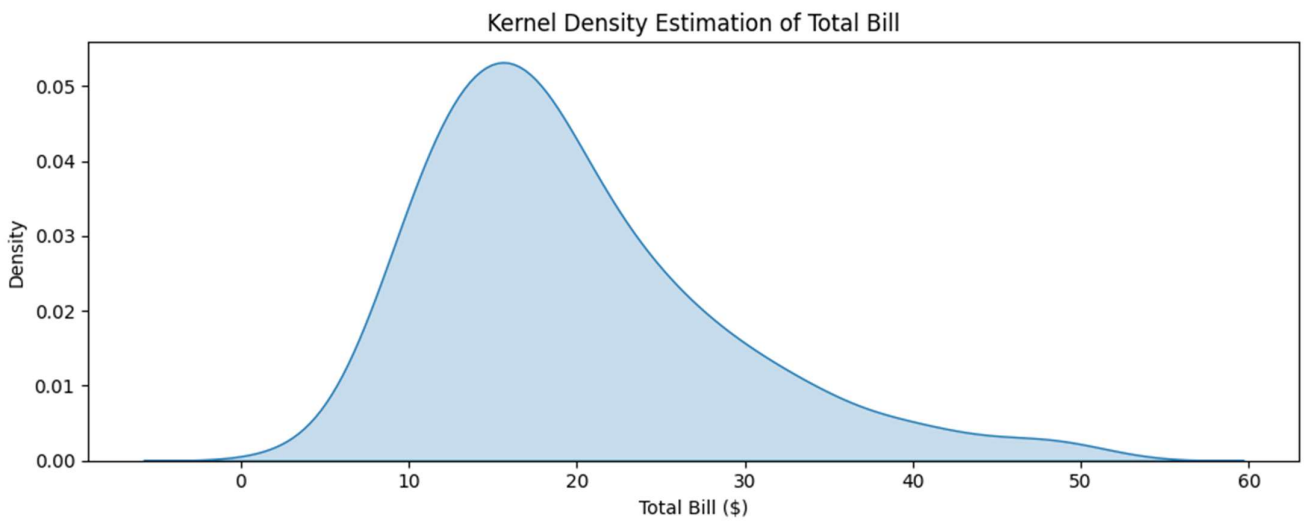

## 3. Univariate data Analysis

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset('tips')
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 12))
sns.histplot(data=tips, x='total_bill', bins=20, kde=True, ax=ax1)
ax1.set_title('Histogram of Total Bill')
ax1.set_xlabel('Total Bill ($)')
ax1.set_ylabel('Frequency')
sns.boxplot(x='total_bill', data=tips, ax=ax2, orient='h', palette='Set1')
ax2.set_title('Quartiles of Total Bill')
ax2.set_xlabel('Total Bill ($)')
```

```
ax2.set_yticks([])
sns.kdeplot(data=tips, x='total_bill', ax=ax3, fill=True)
ax3.set_title('Kernel Density Estimation of Total Bill')
ax3.set_xlabel('Total Bill ($)')
ax3.set_ylabel('Density')
plt.tight_layout()
plt.show()
```



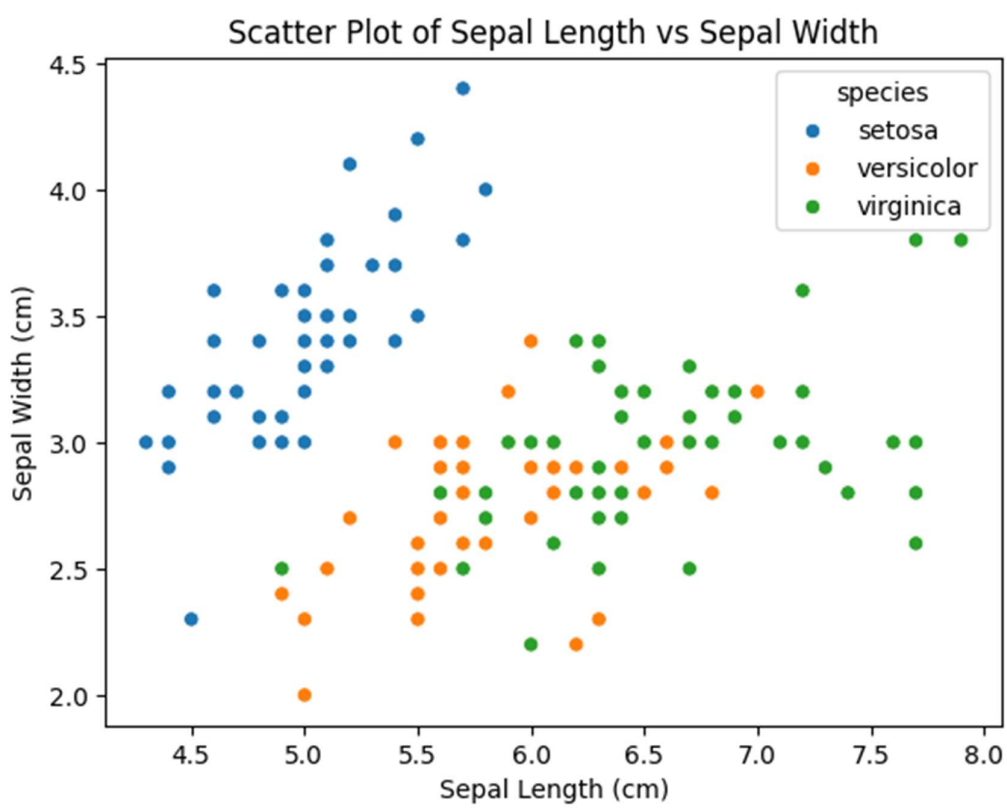



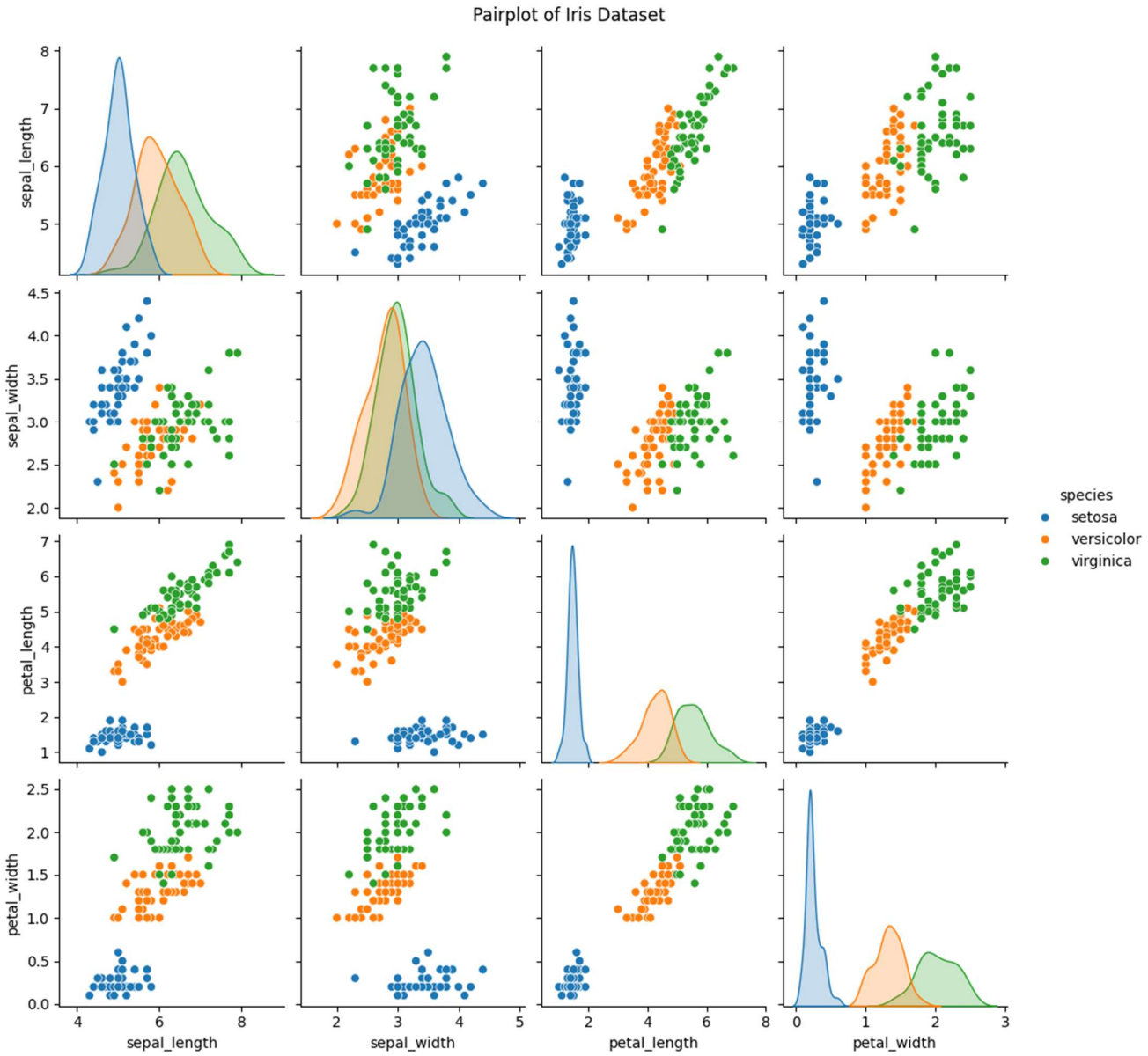



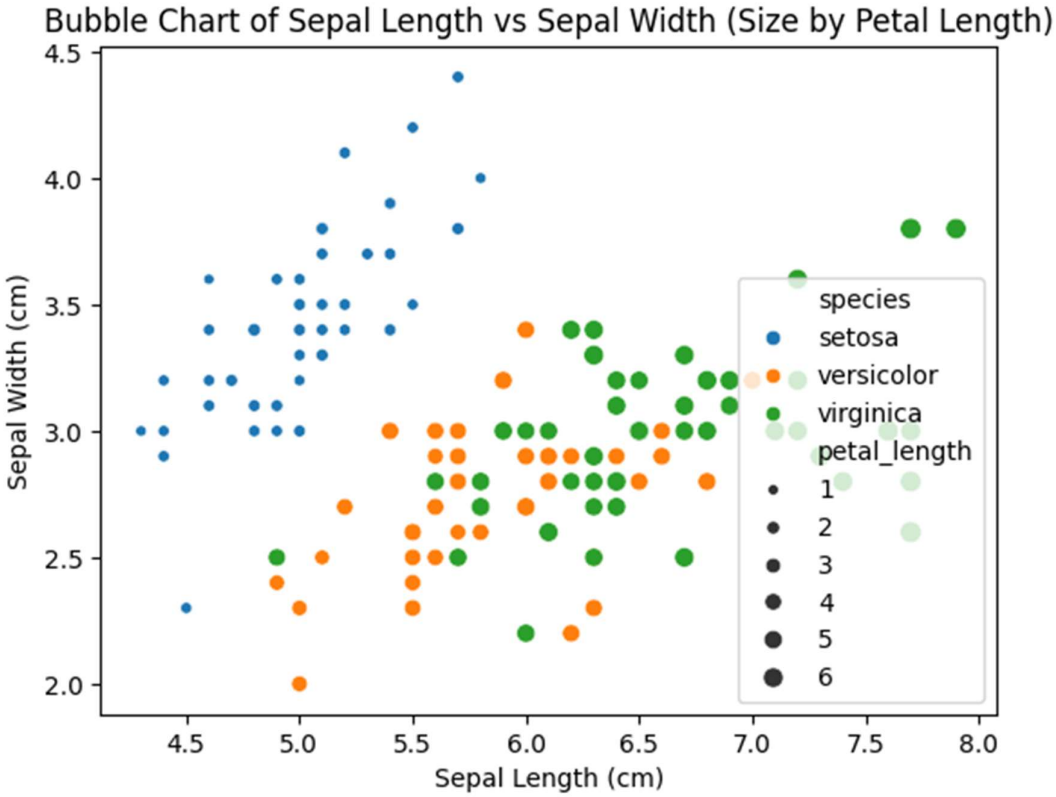

## 4. Multivariate Data Analysis

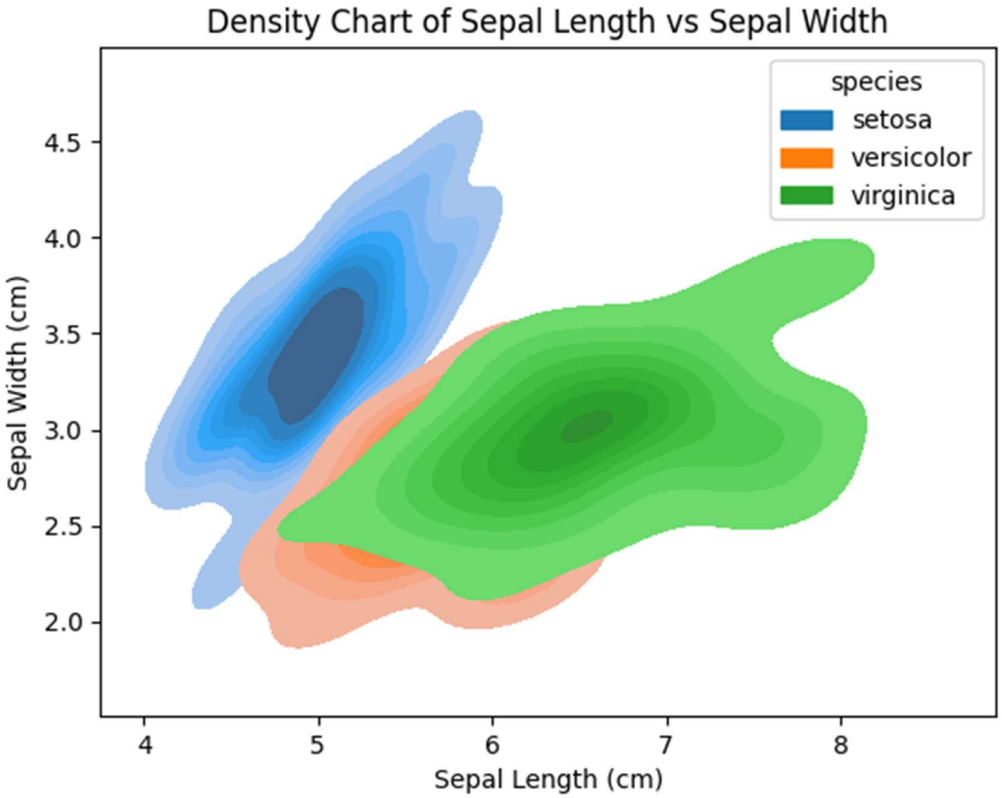
```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
iris = sns.load_dataset('iris')
sns.scatterplot(data=iris, x='sepal_length', y='sepal_width', hue='species')
plt.title('Scatter Plot of Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
sns.pairplot(iris, hue='species')
plt.suptitle('Pairplot of Iris Dataset', y=1.02)
plt.show()
sns.scatterplot(data=iris, x='sepal_length', y='sepal_width', size='petal_length',
hue='species')
plt.title('Bubble Chart of Sepal Length vs Sepal Width (Size by Petal Length)')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
sns.kdeplot(data=iris, x='sepal_length', y='sepal_width', hue='species', fill=True)
plt.title('Density Chart of Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```



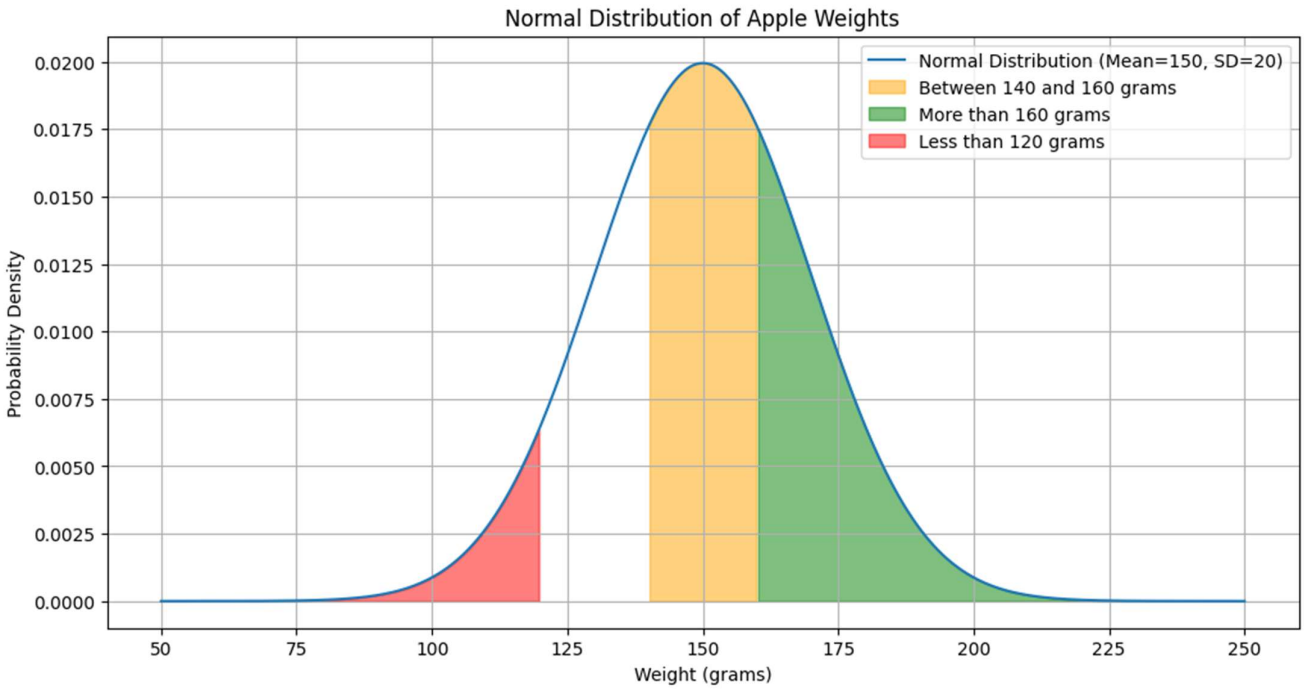

Pairplot of Iris Dataset
sepal_length
sepal_width
petal_length
petal_width
species
setosa
versicolor
virginica


Bubble Chart of Sepal Length vs Sepal Width (Size by Petal Length)
Sepal Width (cm)
Sepal Length (cm)
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
species
setosa
versicolor
virginica
petal_length
1
2
3
4
5
6


Density Chart of Sepal Length vs Sepal Width
Sepal Width (cm)
Sepal Length (cm)
4.5
4.0
3.5
3.0
2.5
2.0
4
5
6
7
8
species
setosa
versicolor
virginica

## 5. Normal Distribution

```
#The weights of apples in a certain orchard follow a normal distribution with a
mean of 150 grams and a standard deviation of 20 grams.
#a) What is the probability that a randomly selected apple weighs between 140 and
160 grams?
#b) What is the probability that a randomly selected apple weighs more than 170
grams?
#c) What is the probability that a randomly selected apple weighs less than 120
grams?

import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm
mean = 150
std_dev = 20
x = np.linspace(50, 250, 500)
y = norm.pdf(x, mean, std_dev)
plt.figure(figsize=(12, 6))
plt.plot(x, y, label='Normal Distribution (Mean=150, SD=20)')
plt.fill_between(x, 0, y, where=((x >= 140) & (x <= 160)), color='orange',
alpha=0.5, label='Between 140 and 160 grams')
plt.fill_between(x, 0, y, where=(x > 160), color='green', alpha=0.5, label='More
than 160 grams')
plt.fill_between(x, 0, y, where=(x < 120), color='red', alpha=0.5, label='Less than
120 grams')
plt.xlabel('Weight (grams)')
plt.ylabel('Probability Density')
plt.title('Normal Distribution of Apple Weights')
plt.legend()
plt.grid(True)
plt.show()
prob_between_140_160 = norm.cdf(160, mean, std_dev) - norm.cdf(140, mean, std_dev)
prob_more_than_170 = 1 - norm.cdf(170, mean, std_dev)
prob_less_than_120 = norm.cdf(120, mean, std_dev)
print(f"Probability of a randomly selected apple weighing between 140 and 160
grams: {prob_between_140_160:.4f}")
print(f"Probability of a randomly selected apple weighing more than 170 grams:
{prob_more_than_170:.4f}")
print(f"Probability of a randomly selected apple weighing less than 120 grams:
{prob_less_than_120:.4f}")
```

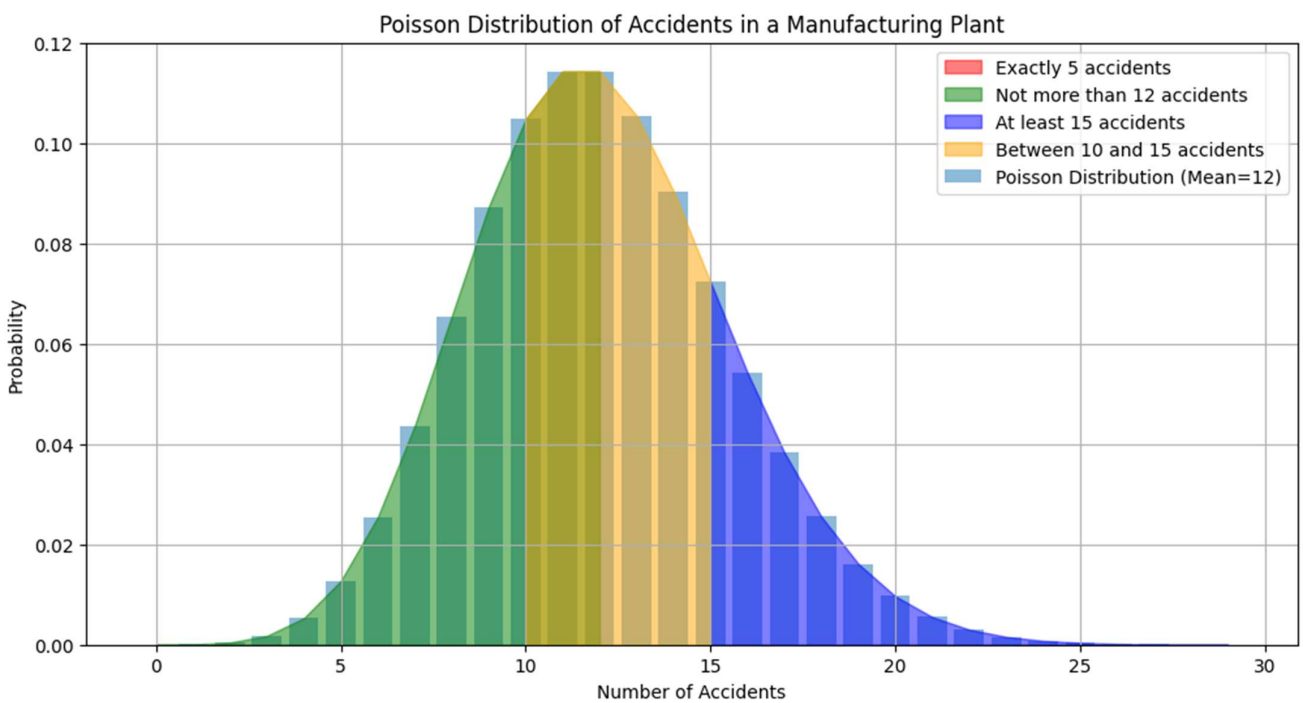



```
Probability of a randomly selected apple weighing between 140 and 160 grams: 0.3829
Probability of a randomly selected apple weighing more than 170 grams: 0.1587
Probability of a randomly selected apple weighing less than 120 grams: 0.0668
```

## 6. Poisson Distribution

```
#The annual number of industrial accidents occurring in a particular manufacturing
plant is known to
# follow Poisson distribution with mean 12.
#a)What is the probability of observing exactly 5 accidents at this plant during
the coming year?
#b)What is the probability of observing not more than 12 accidents at this plant
the coming year?
#c)What is the probability of observing at least 15 accidents at this plant during
the coming year?
#d)What is the probability of observing between 10 and 15 accidents (inclusive) at
this plant during the coming year?

from scipy.stats import poisson
import matplotlib.pyplot as plt
import numpy as np
mean = 12
x = np.arange(0, 30)
poisson_dist = poisson.pmf(x, mean)
plt.figure(figsize=(12, 6))
plt.bar(x, poisson_dist, label='Poisson Distribution (Mean=12)', alpha=0.5)
plt.fill_between(x, 0, poisson_dist, where=(x == 5), color='red', alpha=0.5,
label='Exactly 5 accidents')
```

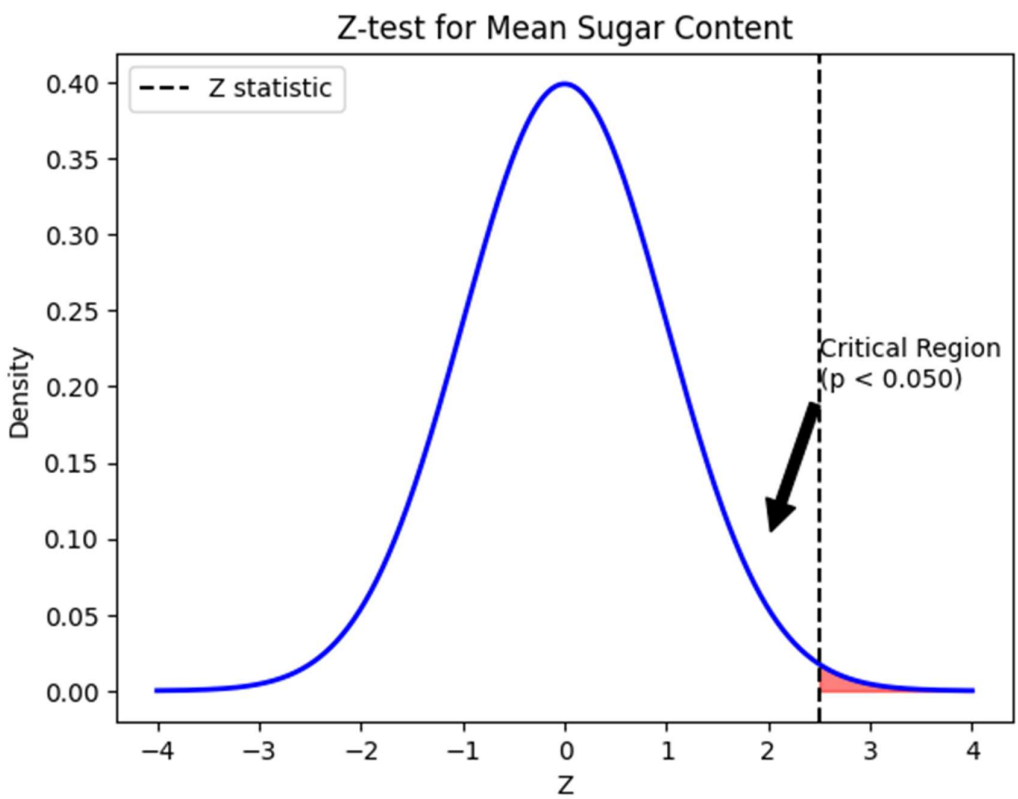
```
plt.fill_between(x, 0, poisson_dist, where=(x <= 12), color='green', alpha=0.5,
label='Not more than 12 accidents')
plt.fill_between(x, 0, poisson_dist, where=(x >= 15), color='blue', alpha=0.5,
label='At least 15 accidents')
plt.fill_between(x, 0, poisson_dist, where=((x >= 10) & (x <= 15)), color='orange',
alpha=0.5, label='Between 10 and 15 accidents')
plt.xlabel('Number of Accidents')
plt.ylabel('Probability')
plt.title('Poisson Distribution of Accidents in a Manufacturing Plant')
plt.legend()
plt.grid(True)
plt.show()
prob_5_accidents = poisson.pmf(5, mean)
print(f"Probability of observing exactly 5 accidents: {prob_5_accidents}")
prob_not_more_than_12 = poisson.cdf(12, mean)
print(f"Probability of observing not more than 12 accidents:
{prob_not_more_than_12}")
prob_at_least_15 = 1 - poisson.cdf(14, mean)
print(f"Probability of observing at least 15 accidents: {prob_at_least_15}")
prob_between_10_and_15 = poisson.cdf(15, mean) - poisson.cdf(9, mean)
print(f"Probability of observing between 10 and 15 accidents:
{prob_between_10_and_15}")
```




```
Probability of observing exactly 5 accidents: 0.012740638735861376
Probability of observing not more than 12 accidents: 0.5759652485730645
Probability of observing at least 15 accidents: 0.22797546769645516
Probability of observing between 10 and 15 accidents: 0.6020234907796708
```

## 7. Z-Test

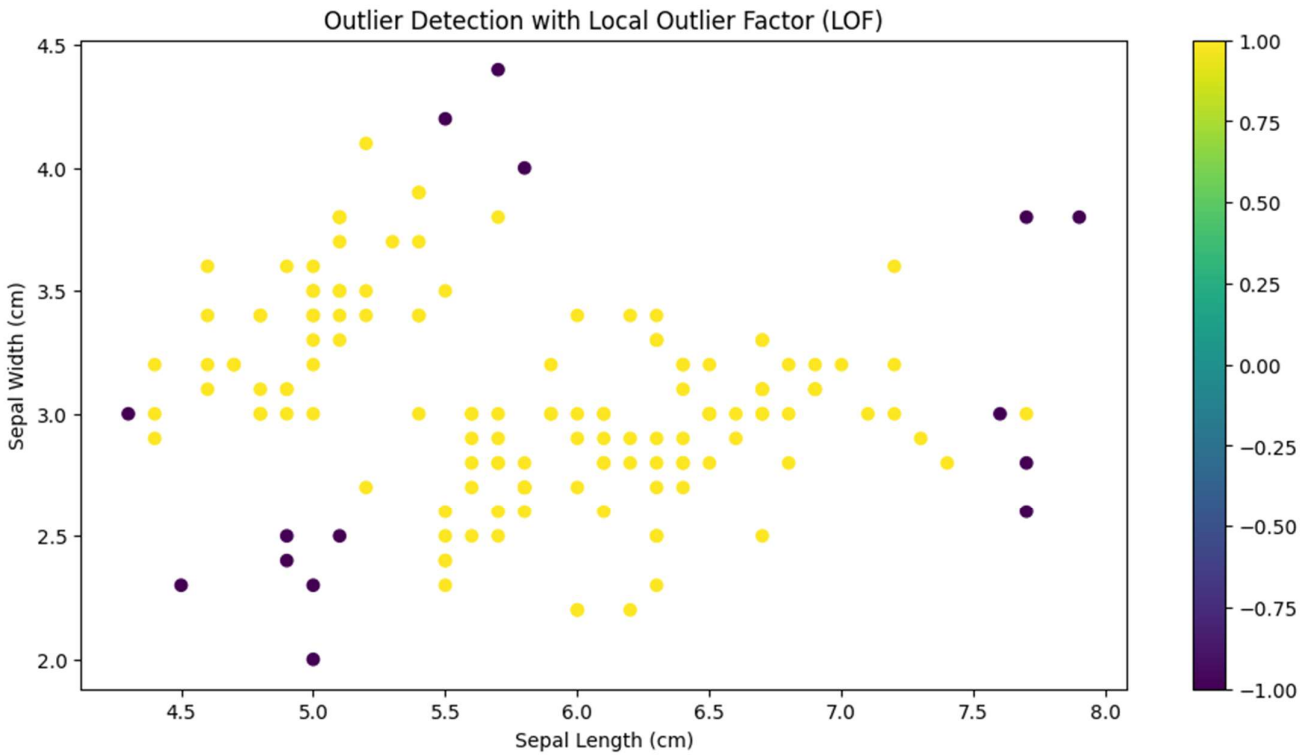
```
# Question: A soft drink company claims that the mean sugar content in its soda
bottles is 40 grams.
# To test this claim, a random sample of 25 bottles is selected, and the sugar
content is measured.
# The sample mean is found to be 38 grams with a standard deviation of 4 grams.
# Conduct a Z-test at a significance level of 0.05 to determine if there is enough
evidence to reject the company's claim.

import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
sample_mean = 38
pop_mean = 40
sample_std = 4
sample_size = 25
alpha = 0.05
z_statistic = (sample_mean - pop_mean) / (sample_std / np.sqrt(sample_size))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
x = np.linspace(-4, 4, 1000)
y = stats.norm.pdf(x, 0, 1)
plt.plot(x, y, 'b-', linewidth=2)
shade = np.linspace(abs(z_statistic), 4, 300)
plt.fill_between(shade, stats.norm.pdf(shade, 0, 1), color='red', alpha=0.5)
plt.title('Z-test for Mean Sugar Content')
plt.xlabel('Z')
plt.ylabel('Density')
plt.annotate('Critical Region\n(p < {:.3f})'.format(alpha), xy=(2, 0.1),
xytext=(2.5, 0.2),
             arrowprops=dict(facecolor='black', shrink=0.05))
plt.axvline(x=abs(z_statistic), color='black', linestyle='--', label='Z statistic')
plt.legend()
plt.show()
if p_value < alpha:
    print("Reject the null hypothesis: There is enough evidence to conclude that
the mean sugar content is not 40 grams.")
else:
    print("Fail to reject the null hypothesis: There is not enough evidence to
conclude that the mean sugar content is not 40 grams.")
```




```
Reject the null hypothesis: There is enough evidence to conclude that the mean
sugar content is not 40 grams.
```

## 8. Outlier Detection

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.neighbors import LocalOutlierFactor
iris = load_iris()
X = iris.data
y = iris.target
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1)
y_pred = lof.fit_predict(X)
plt.figure(figsize=(12, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
plt.colorbar()
plt.title('Outlier Detection with Local Outlier Factor (LOF)')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```




## 9. Performance Evaluation

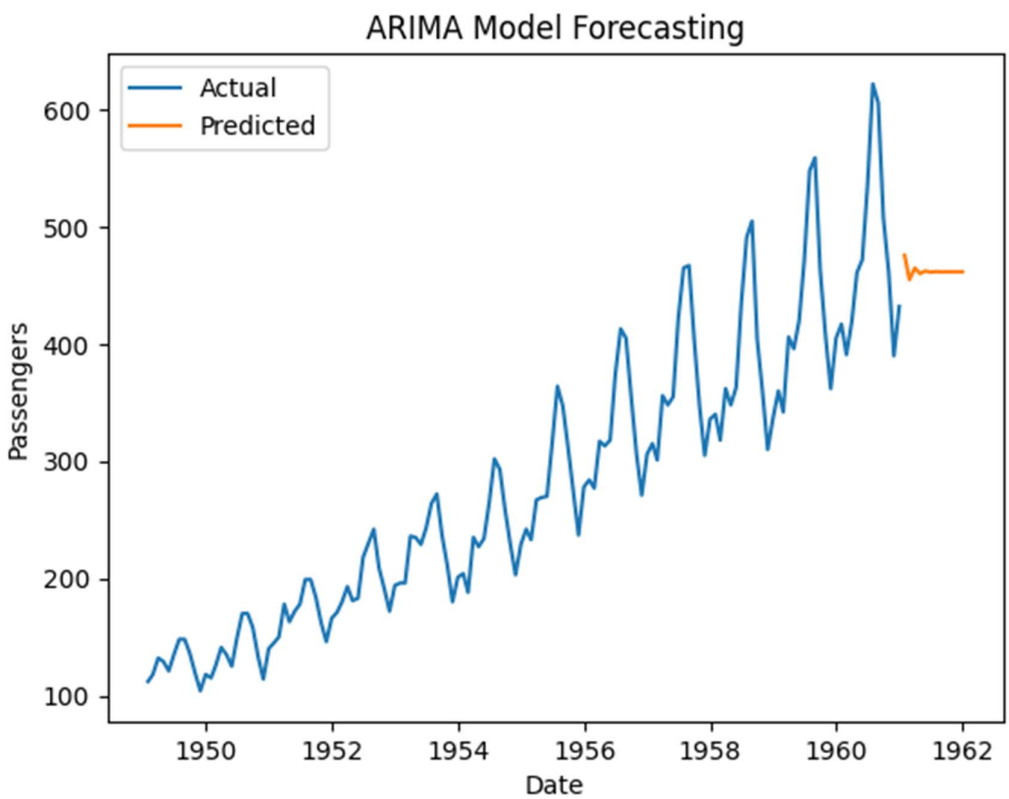
```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
mean_absolute_percentage_error
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mape = mean_absolute_percentage_error(y_test, y_pred)
mase = mae / (sum(abs(y_test[i + 1] - y_test[i]) for i in range(len(y_test) - 1)) /
(len(y_test) - 1))
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Square Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.2f}")
print(f"Mean Absolute Scaled Error (MASE): {mase:.2f}")
```

Output:

```
Mean Absolute Error (MAE): 42.79
Root Mean Square Error (RMSE): 53.85
Mean Absolute Percentage Error (MAPE): 0.37
Mean Absolute Scaled Error (MASE): 0.53
```

## 10. ARIMA

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
from math import sqrt
from statsmodels.datasets import get_rdataset
air_passengers = get_rdataset('AirPassengers', 'datasets').data['value']
air_passengers.index = pd.date_range(start='1949-01-01',
periods=len(air_passengers), freq='M')
model = ARIMA(air_passengers, order=(1, 1, 1))
model_fit = model.fit()
predictions = model_fit.predict(start=len(air_passengers),
end=len(air_passengers)+11, typ='levels')
plt.plot(air_passengers.index, air_passengers, label='Actual')
plt.plot(predictions.index, predictions, label='Predicted')
plt.title('ARIMA Model Forecasting')
plt.xlabel('Date')
plt.ylabel('Passengers')
plt.legend()
plt.show()
rmse = sqrt(mean_squared_error(air_passengers[-12:], predictions))
print(f'Root Mean Squared Error (RMSE): {rmse:.2f}')
```




```
Root Mean Squared Error (RMSE): 76.29
```